Name(s):	**Student ID(s):**
Daniela Alejandra Escamilla Alanis Leonardo Alonso González del Ángel Dana Elena Zertuche Castro	T03086448 T03098643 T02991408
Subject:	**Teacher:**
Knowledge Management	Alfredo Paul Aguirre Torres
Homework:	**Date:**
Project Progress - Knowledge Management System & Prototypes	23/10/2026

PolloSoft: Knowledge Management System

Introduction

For this project we created a fictional company called **PolloSoft** and designed a knowledge management system for it. In this progress report we describe the company and the knowledge it handles, we explain its knowledge lifecycle, we present an ontology of its main concepts and documents, and we show a first prototype made in Google Colab. The prototype can classify documents automatically and search documents using questions written in natural language.

1. Description of the Fictional Company

PolloSoft is a small technology company from Monterrey, Nuevo León, that develops native apps for iPhone and iPad. The team uses Swift and SwiftUI and follows a *local-first* approach: the user's data is saved on the device first and then synced with iCloud, so the apps also work without internet. PolloSoft has three apps: **NestNotes** (notes and journaling), **Featherweight** (habit tracker) and **RoosterFocus** (focus timer). The apps make money with monthly and annual subscriptions.

Type of company: private technology company / independent iOS app studio.

Core activities:

- **Native iOS app development** with Swift, SwiftUI, SwiftData and CloudKit.
- **Automated QA testing** (end-to-end UI tests with XCUITest) to find bugs before each release.
- **Subscription and monetization management** with StoreKit 2 and App Store Optimization (ASO).
- **Organic social media marketing** (TikTok, Instagram Reels and YouTube Shorts) to get more downloads.

PolloSoft also has **tacit knowledge** that is only in people's heads, for example how to solve an App Review rejection or which type of video works better on TikTok (Nonaka & Takeuchi, 1995). One goal of the knowledge management system is to turn this tacit knowledge into **explicit** documents that everyone can find and use.

Types of knowledge and documents: PolloSoft handles six main types of documents. These are also the six categories that our prototype uses to classify documents.

Table 1. Types of documents at PolloSoft

Document type	What it contains	Example
Technical Guide	How the apps are built (architecture, code rules)	SwiftData migration strategy
Deployment SOP	Step-by-step procedures to publish the apps	SOP: submit a build to App Store Connect
Bug Report	Bugs found and results of QA tests	Bug: crash after update
Analytics Report	Revenue (ARR), crashes, retention and downloads	Revenue report September
Marketing Plan	Social media, ASO and campaigns	Q4 social media plan
Policy	Privacy, security and onboarding rules	Data privacy policy

2. Knowledge Lifecycle Map

Based on the knowledge management cycle (Dalkir, 2017), we defined six stages for PolloSoft. The stages form a cycle around a shared knowledge base, because knowledge must be updated all the time (for example, every year Apple announces new tools at WWDC and some guides become outdated).



Figure 1. Knowledge lifecycle map of PolloSoft.

Table 2. Stages of the knowledge lifecycle

Stage	What happens at PolloSoft	Who / tools
1. Creation / Acquisition	Developers write technical guides, QA writes bug reports, the product team exports analytics from App Store Connect, and the team writes lessons learned after each release.	All teams; templates in Google Docs
2. Storage / Documentation	Every document is saved in a shared Google Drive folder with its owner, category, version and review date.	Document owner; Google Drive
3. Organization / Classification	The prototype classifies each new document into one of the 6 categories. The ontology connects documents with concepts (for example, a bug report with "Crash").	Our classifier (Google Colab)
4. Sharing	Employees find documents with the semantic search. New knowledge is also shared in weekly meetings and in the onboarding of new employees.	Semantic search, meetings
5. Application	The team follows the SOPs in every release, uses bug reports to fix problems and uses analytics reports to decide marketing and prices.	All teams
6. Update / Improvement	Documents are reviewed every 6 months and after WWDC. Old documents are marked as outdated. The classifier is trained again with the new documents.	Document owner

Example: a QA engineer finds that the app crashes after an update (*creation*). She writes a bug report and saves it in the shared folder (*storage*). The classifier labels it as a Bug Report (*organization*). A developer searches "the app crashes after the update" and finds it (*sharing*). The team fixes the bug following the hotfix SOP (*application*), and the migration guide is updated so the problem does not happen again (*update*).

3. Ontology of Organizational Knowledge

An ontology is a formal description of the concepts of a domain and the relationships between them (Gruber, 1993). Our ontology has two types of elements: the **document types** of PolloSoft and the **concepts** that those documents talk about. We wrote the relationships as *(concept, relationship, concept)* and drew them in the notebook with the networkx library.

3.1 Main concepts

Table 3. Main concepts of the ontology

Group	Concepts
Product and technology	iOS App, SwiftUI, SwiftData, CloudKit, StoreKit 2
Release	Build, App Store Connect, TestFlight
Quality	Bug, Crash, UI Tests
Business	Subscription, Revenue (ARR), Churn, Downloads
Marketing	Social Media, ASO
Governance	Privacy, User Data

3.2 Relationships between concepts

Table 4. Relationships between concepts

Concept	Relationship	Concept	Concept	Relationship	Concept
iOS App	*built with*	SwiftUI	Build	*tested in*	TestFlight
iOS App	*stores data in*	SwiftData	UI Tests	*find*	Bug
SwiftData	*syncs with*	CloudKit	Crash	*is a type of*	Bug
iOS App	*sells*	Subscription	Social Media	*brings*	Downloads
Subscription	*uses*	StoreKit 2	ASO	*brings*	Downloads
Subscription	*generates*	Revenue (ARR)	Downloads	*convert into*	Subscription
Churn	*reduces*	Revenue (ARR)	Privacy	*protects*	User Data
Build	*uploaded to*	App Store Connect			

3.3 Relationships between documents and concepts

Table 5. Relationships between document types and concepts

Document type	Relationship	Concepts
Technical Guide	*describes*	SwiftUI, SwiftData
Deployment SOP	*explains*	Build, App Store Connect
Bug Report	*documents*	Bug, Crash
Analytics Report	*measures*	Revenue (ARR), Churn, Crash
Marketing Plan	*plans*	Social Media, ASO
Policy	*defines*	Privacy

Figure 2 shows the complete ontology. The blue nodes are the document types and the orange nodes are the concepts. For example, it shows that a *Bug Report documents a Crash*, that a *Crash is a type of Bug*, and that *Downloads convert into Subscriptions* that *generate Revenue*. This helps to see which documents are needed to understand each concept.

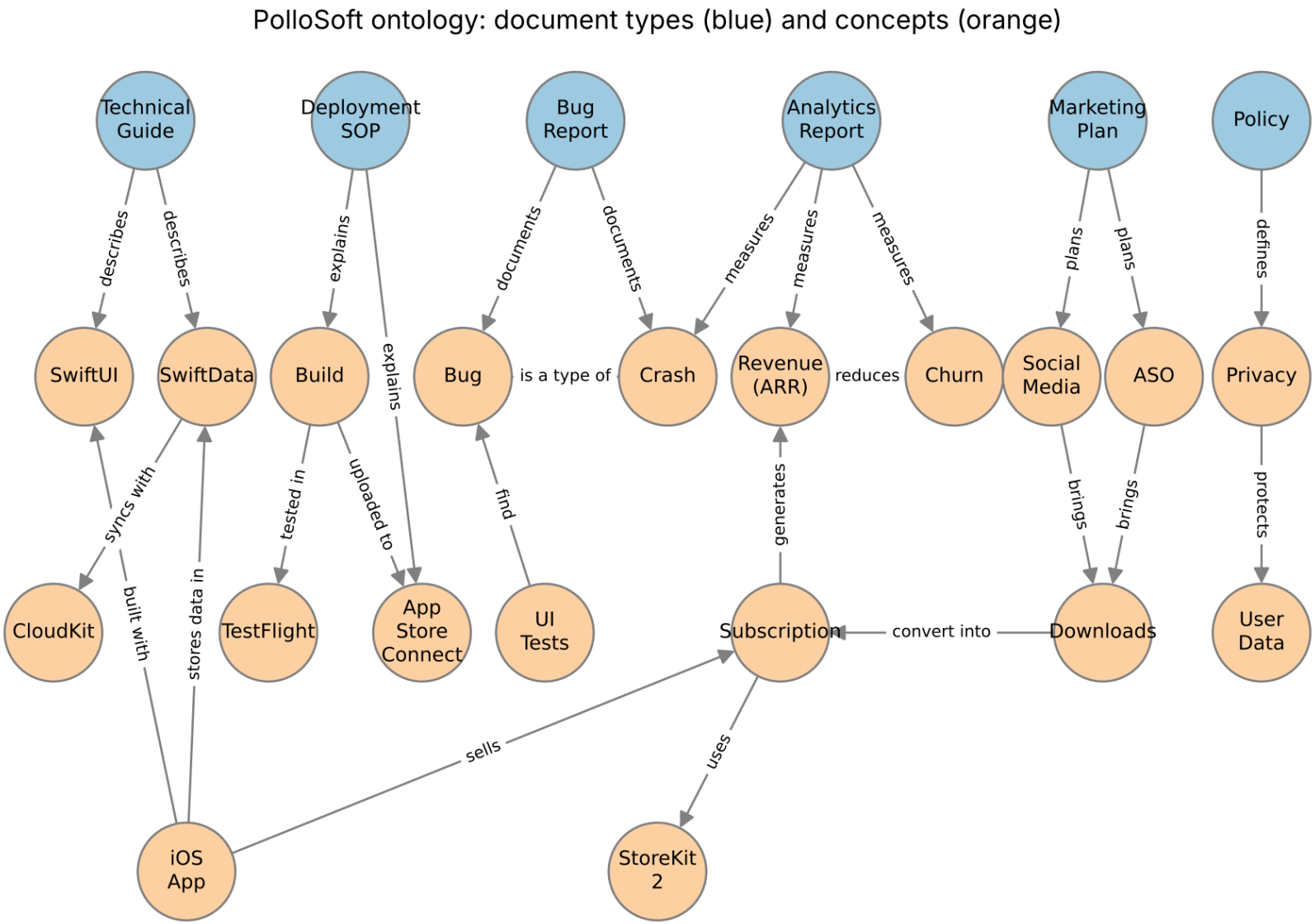


Figure 2. Ontology of PolloSoft (output of the Colab notebook).

4. Explanation of the Developed Prototype

We developed the prototype in **Google Colab** with Python. We used **pandas** to organize the data, **scikit-learn** for TF-IDF, the classifiers and cosine similarity (Pedregosa et al., 2011), **matplotlib** for the charts and **networkx** for the ontology graph. The notebook is *PolloSoft_KM_Prototype.ipynb* and it runs completely with *Runtime → Run all*.

4.1 Dataset

We wrote **48 short mock documents**, 8 for each of the 6 categories in Table 1. Each document has a title, a text and its category. The title and the text are joined into one text for the analysis.

4.2 Text representation with TF-IDF

To work with text, the documents are converted into numbers with **TF-IDF** (Term Frequency - Inverse Document Frequency). TF counts how many times a word appears in a document, and IDF gives less importance to words that appear in many documents. Specific words like *TestFlight* or *churn* get a higher weight than common words like *app*. We also changed the text to lowercase and removed English stop words ("the", "and", "is"). With all the documents, the vocabulary has **456 different words**.

4.3 Automatic classification

We divided the documents into 75% for training (36 documents) and 25% for testing (12 documents). We tested two classifiers that are common for text: **Multinomial Naive Bayes** and **Logistic Regression**. Each model is a pipeline (first TF-IDF, then the classifier). We compared them with the accuracy on the test set and with **5-fold cross-validation**, which trains and tests the model five times with different parts of the data. After that, the best model is used to predict the category of new documents.

```
models = {
    "Naive Bayes": make_pipeline(TfidfVectorizer(stop_words="english"), MultinomialNB()),
    "Logistic Regression": make_pipeline(TfidfVectorizer(stop_words="english"),
                                         LogisticRegression(C=10, max_iter=1000)),
}
for name, model in models.items():
    model.fit(X_train_text, y_train)
    test_accuracy = accuracy_score(y_test, model.predict(X_test_text))
    cv_scores = cross_val_score(model, df["full_text"], df["category"], cv=5)
```

4.4 Semantic search

For the search, the question of the user is converted into a TF-IDF vector with the same vectorizer of the documents. Then we calculate the **cosine similarity** between the question and every document. The result goes from 0 (no words in common) to 1 (very similar), and the documents with the highest score are shown first (Manning et al., 2008).

```
def search(query, top_n=3):
    query_vector = vectorizer.transform([query])            # same vectorizer used for the documents
    scores = cosine_similarity(query_vector, X_all)[0]       # one score per document
    best = scores.argsort()[::-1][:top_n]                     # positions of the highest scores
    return pd.DataFrame({
        "title": df.loc[best, "title"].values,
        "category": df.loc[best, "category"].values,
        "similarity score": scores[best].round(3),
    })
```

4.5 Ontology graph

The relationships of Section 3 are saved in a list in the notebook. With networkx we create a directed graph where each relationship is an arrow with a label, and we draw it with different colors for document types and concepts (Figure 2).

5. Evidence of System Operation and Execution Examples

5.1 Mock documents

All the results in this section are outputs of the Colab notebook. Table 6 shows one of the mock documents of each category.

Table 6. Example of one document per category

Title	Category	Text
NestNotes architecture	Technical Guide	NestNotes saves every note locally with SwiftData first and then syncs it to iCloud using CloudKit, so the app works offline.
SOP: submit a build to App Store Connect	Deployment SOP	Step 1: increase the build number. Step 2: archive the app in Xcode. Step 3: upload it to App Store Connect and submit it for review.
Bug: crash after update	Bug Report	The app crashes on launch after updating to version 2.2. Steps to reproduce: install 2.1, create notes, update and open the app. Severity: critical.
Revenue report September	Analytics Report	ARR reached 48,200 USD this month, 6 percent more than last month. MRR was 4,017 USD and monthly churn stayed at 5.2 percent.
Q4 social media plan	Marketing Plan	Grow organic installs 25 percent with TikTok, Instagram Reels and YouTube Shorts. Post five videos per week with a strong hook.
Data privacy policy	Policy	We collect the minimum personal data. Notes stay on the device or in the user's iCloud and we do not use tracking SDKs.

5.2 Classification results

Table 7. Comparison of the two classifiers

Model	Test accuracy	Cross-validation accuracy (mean)
Naive Bayes	0.833	0.707
Logistic Regression	0.917	0.787

Logistic Regression had the best results in both tests (91.7% on the test set and 78.7% with cross-validation, compared to 83.3% and 70.7% for Naive Bayes), so we selected it. The cross-validation result is lower because it uses different splits of a small dataset, so it is a more realistic estimate.

Table 8. Classification report on the test set (Logistic Regression)

Category	Precision	Recall	F1-score	Documents
Analytics Report	1.00	1.00	1.00	2
Bug Report	1.00	0.50	0.67	2
Deployment SOP	1.00	1.00	1.00	2
Marketing Plan	0.67	1.00	0.80	2
Policy	1.00	1.00	1.00	2
Technical Guide	1.00	1.00	1.00	2
Accuracy			**0.92**	12

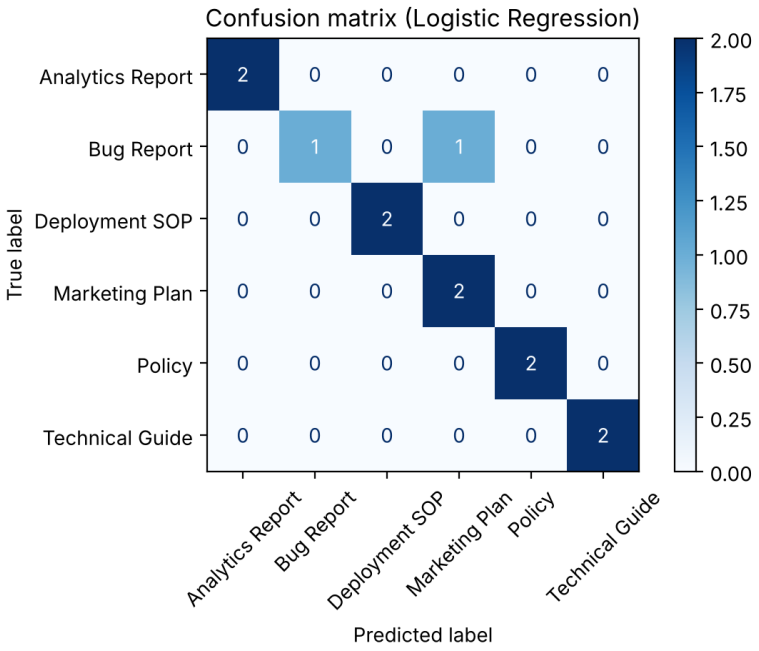


Figure 3. Confusion matrix of Logistic Regression on the 12 test documents.

Table 9. Predictions for the test documents

Test document	True category	Predicted category	Correct?
SOP: phased release	Deployment SOP	Deployment SOP	Yes
Bug: purchase button not working on iPad	Bug Report	Bug Report	Yes
Swift concurrency rules	Technical Guide	Technical Guide	Yes
App Store conversion report	Analytics Report	Analytics Report	Yes
Data breach policy	Policy	Policy	Yes
ASO keyword research	Marketing Plan	Marketing Plan	Yes
RoosterFocus 2.0 launch campaign	Marketing Plan	Marketing Plan	Yes
SOP: hotfix release	Deployment SOP	Deployment SOP	Yes
E2E test plan for onboarding	Bug Report	Marketing Plan	**No**
SwiftUI MVVM guide	Technical Guide	Technical Guide	Yes
Remote work policy	Policy	Policy	Yes
Weekly KPI summary	Analytics Report	Analytics Report	Yes

The model classified 11 of 12 test documents correctly. The error was *"E2E test plan for onboarding"*, a Bug Report that was predicted as Marketing Plan. This document is a test plan and it does not have the typical words of a bug report like *bug*, *severity* or *steps to reproduce*, so it was harder to recognize.

5.3 Classification of new documents

We tested the model with six new documents that were not part of the dataset:

Table 10. Predictions for new documents

New document	Expected	Predicted	Probability
The app freezes when I tap restore purchases on iPhone 13. Steps to reproduce: open settings and tap restore. Severity: high.	Bug Report	Bug Report	0.77
Upload version 2.3 with Transporter, select the build in App Store Connect and submit it for review.	Deployment SOP	Deployment SOP	0.66
October ARR was 51,100 USD and monthly churn went down to 4.8 percent.	Analytics Report	Analytics Report	0.65
Plan daily TikTok videos about new year goals and update the App Store keywords for January.	Marketing Plan	Marketing Plan	0.47
From now on two-factor authentication is mandatory for all employee accounts in GitHub and App Store Connect.	Policy	Policy	0.27
We will replace the CloudKit mirroring with CKSyncEngine and keep SwiftData as the local database.	Technical Guide	Technical Guide	0.58

The model predicted 6 of the 6 new documents correctly. The probability shows how sure the model is. The lowest one was for the Policy document (0.27), because it has words that also appear in other categories (like GitHub and App Store Connect). In a real system, documents with a low probability could be reviewed by a person before saving them.

5.4 Semantic search examples

These are the top 3 results of four questions written in natural language:

Table 11. Search results for: "the app crashes after the update"

#	Document	Category	Similarity score
1	Bug: crash after update	Bug Report	0.552
2	ASO keyword research	Marketing Plan	0.185
3	Weekly crash report	Analytics Report	0.144

Table 12. Search results for: "how do I submit the app to the App Store"

#	Document	Category	Similarity score
1	SOP: submit a build to App Store Connect	Deployment SOP	0.537
2	SOP: in-app purchase setup	Deployment SOP	0.328
3	App Store screenshots test	Marketing Plan	0.285

Table 13. Search results for: "why do users cancel their subscription"

#	Document	Category	Similarity score
1	Cancellation survey results	Analytics Report	0.467
2	Weekly KPI summary	Analytics Report	0.164
3	Bug: restore purchases fails	Bug Report	0.108

Table 14. Search results for: "what should a new developer do the first week"

#	Document	Category	Similarity score
1	New developer onboarding checklist	Policy	0.513
2	November content calendar	Marketing Plan	0.317
3	Weekly KPI summary	Analytics Report	0.177

In the four examples the most useful document appears in first place with a clearly higher score than the other results. For example, for *"the app crashes after the update"* the first result is the bug report *Bug: crash after update* (0.552). We also noticed a limitation: *ASO keyword research* appears in second place only because it shares the word "update". TF-IDF compares words, not meanings, so it can return documents that are not really related.

Conclusions

In this progress we designed the knowledge management system of PolloSoft: we described the company and its documents, we defined a six-stage knowledge lifecycle and we created an ontology that connects document types with the main concepts of the company. We also developed a working prototype in Google Colab. The classifier (Logistic Regression) identified the category of 92% of the test documents and all of the new documents, and the semantic search found the correct document in first place for every example question.

The main limitation is that the dataset is small and was written by us, so the results are only a demonstration. For the final delivery we want to add more documents, improve the search so it understands synonyms (for example with sentence embeddings), and connect the ontology with the search results.

References

Dalkir, K. (2017). *Knowledge management in theory and practice* (3rd ed.). MIT Press.

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition, 5*(2), 199-220.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Nonaka, I., & Takeuchi, H. (1995). *The knowledge-creating company*. Oxford University Press.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.